

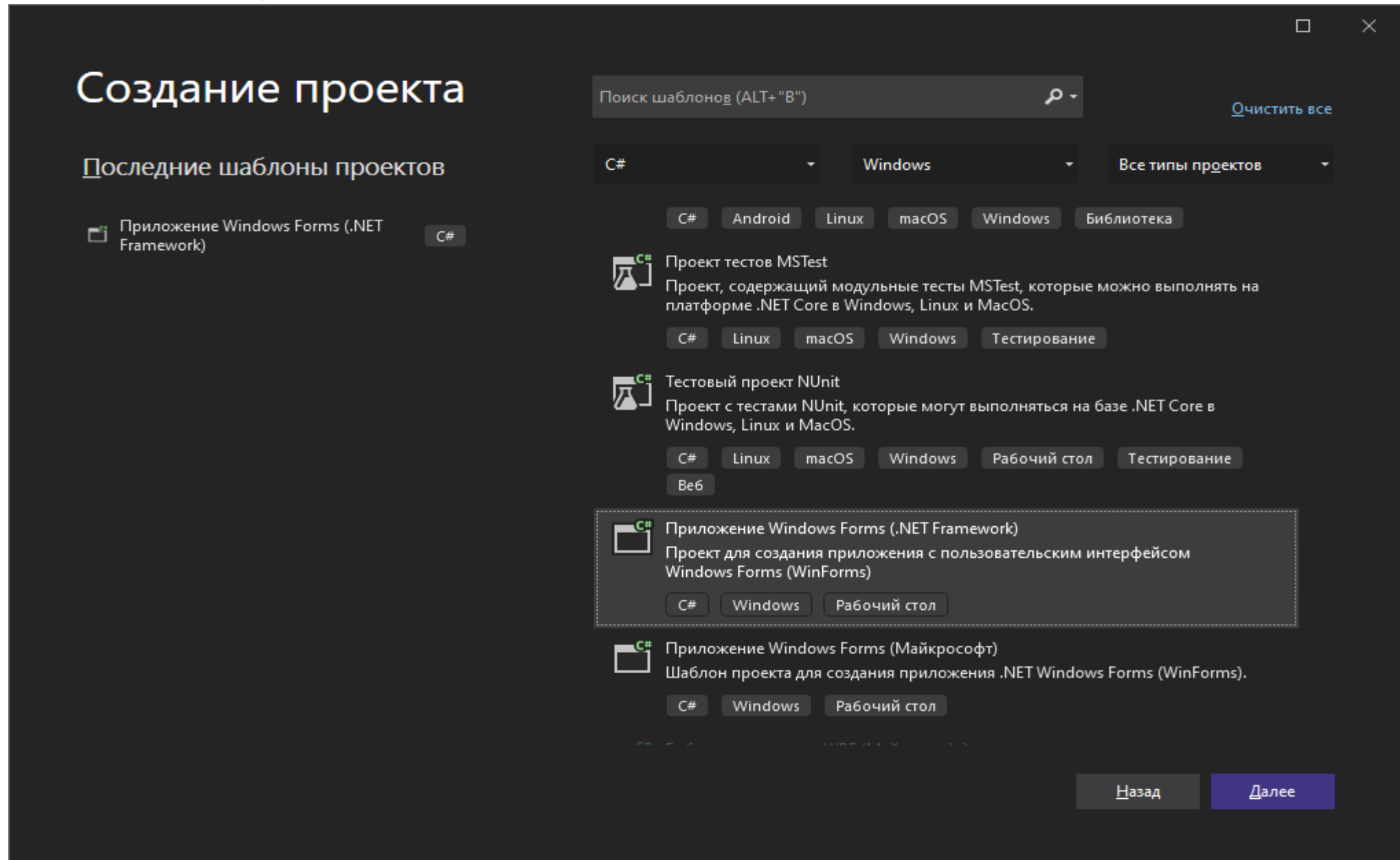
Неуправляемое движение

Мяч, движущийся с переменной скоростью

Какое приложение мы хотим...

- По форме **летает мячик**, при этом мы можем **замедлять** и **ускорять** его движение с помощью стрелочек.
- По кнопке «Start» мы можем запустить движение мячика, по кнопке «Stop» - остановить это движение
- Таким образом, на экране есть 3 элемента управления
 - Кнопка «Start»
 - Кнопка «Stop»
 - Элемент управления со стрелками, который позволяет менять скорость шарика

Создаем проект Windows Forms(.Net Framework)



Проект называем WindowsFormsAppBall

Настроить новый проект

Приложение Windows Forms (.NET Framework) C# Windows Рабочий стол

Имя проекта

WindowsFormsAppBall

Расположение

C:\Users\Administrator\source\repos

Имя решения ⓘ

WindowsFormsAppBall

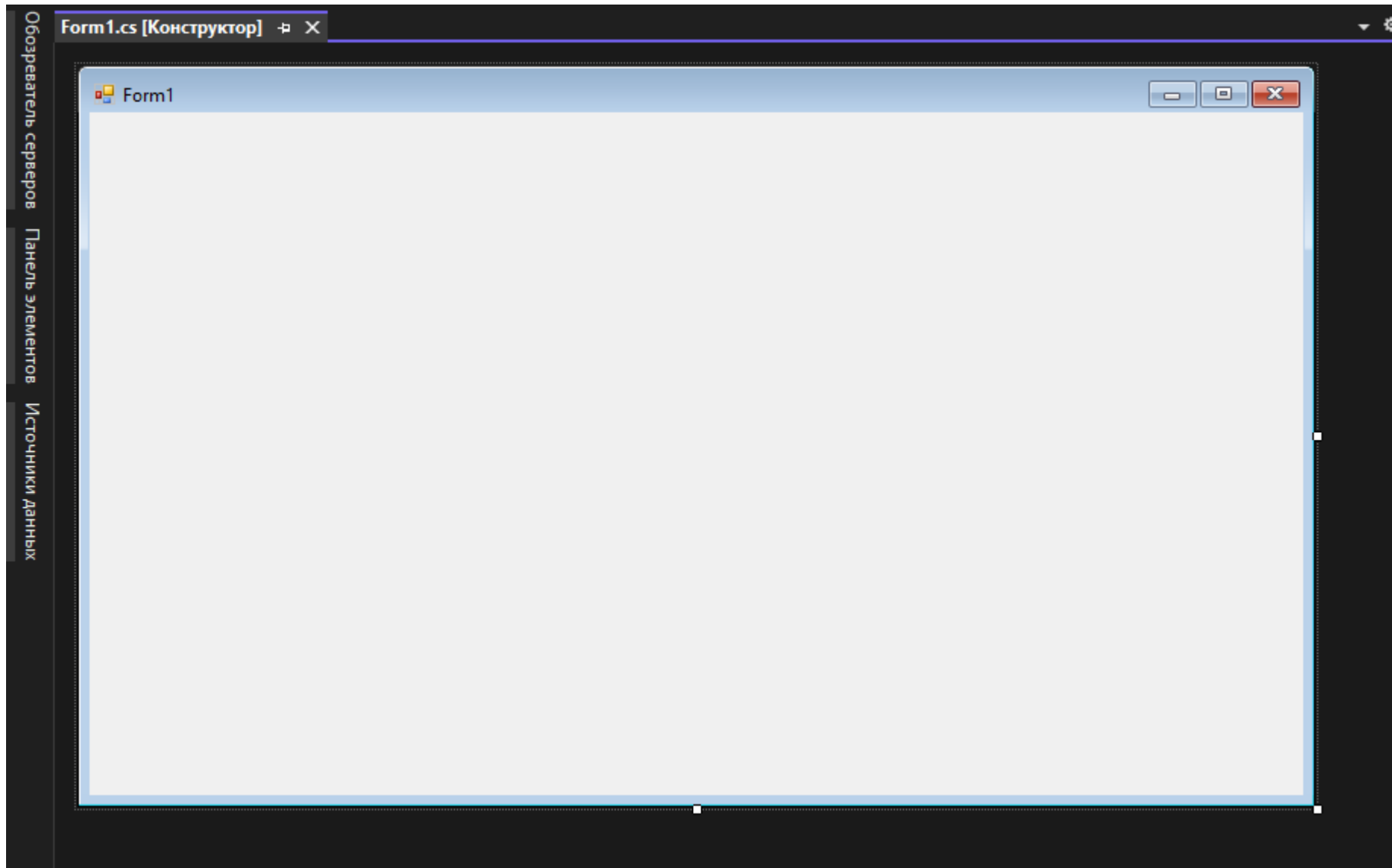
☒ Поместить решение и проект в одном каталоге

Платформа

.NET Framework 4.7.2

Назад Создать

Открывается стандартное окно шаблона проекта



Помещаем на форму элементы управления в той последовательности, как они указаны в списке

- Кнопка **“Start”** (чтобы запускать движение мяча)
- Кнопка **«Stop»** (чтобы останавливать движение мяча)
- Элемент **«Timer»**, который виден ниже формы, но на форме не отображается (на каждый интервал «тиков» Timer (таймера) мяч будет смещаться на 1 шаг по x и по y)
- Элемент **numericUpDown** (чтобы регулировать скорость мяча)

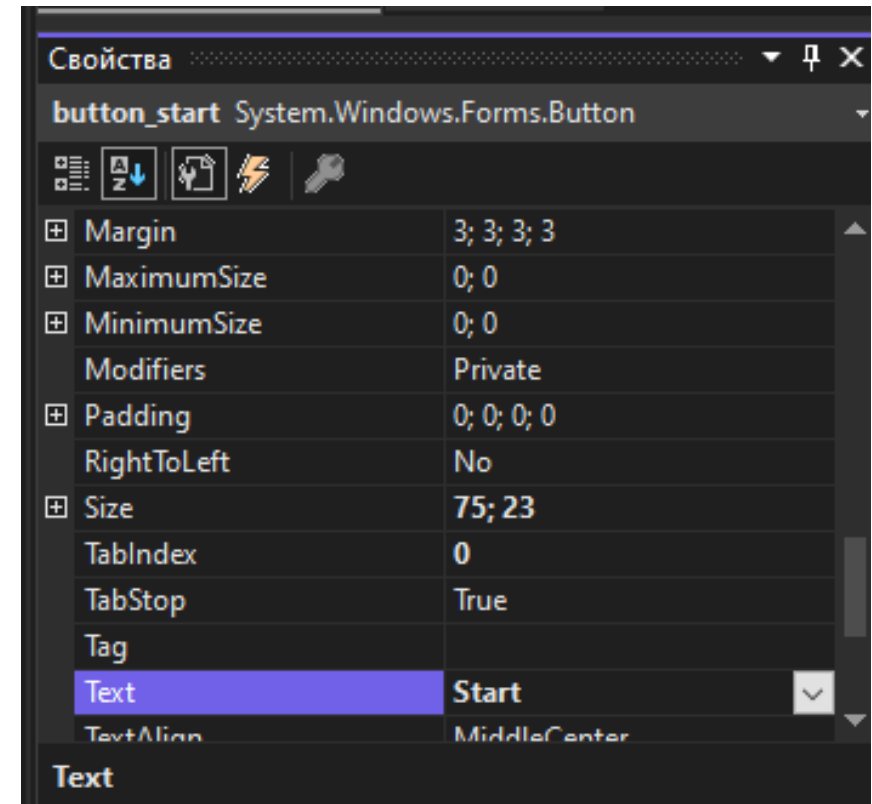
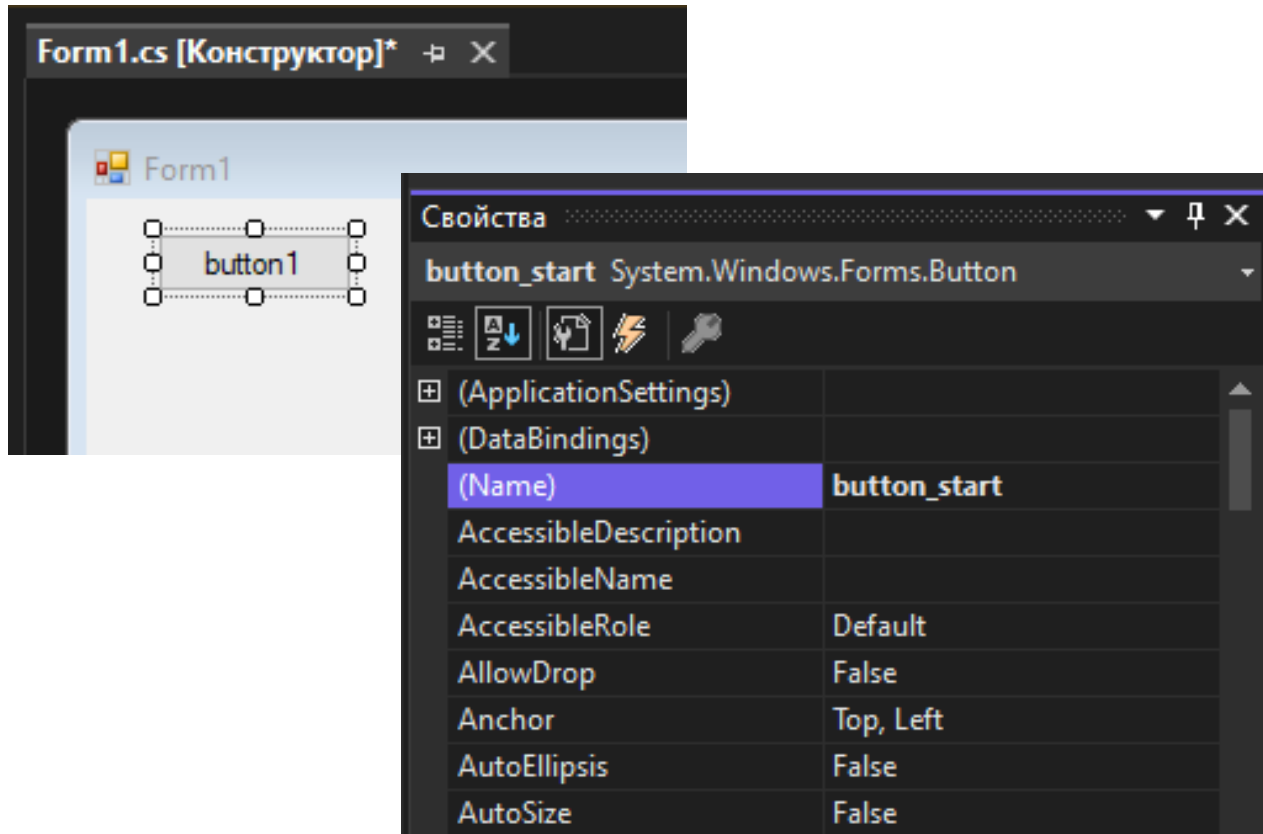
Изменение имен объектов управления

- Когда мы создаем кнопки, система по умолчанию дает им свои имена `button1` и `button2`. Для более ясного чтения текста программы заменим их на `button_start` и `button_stop`
- Для этого свойство `Name` изменим с `button1(button2)` на `button_start (button_stop)`
- Свойство `Text` с `button1 (button2)` заменим соответственно на `Start (Stop)`
- Технология замены показана на следующем слайде

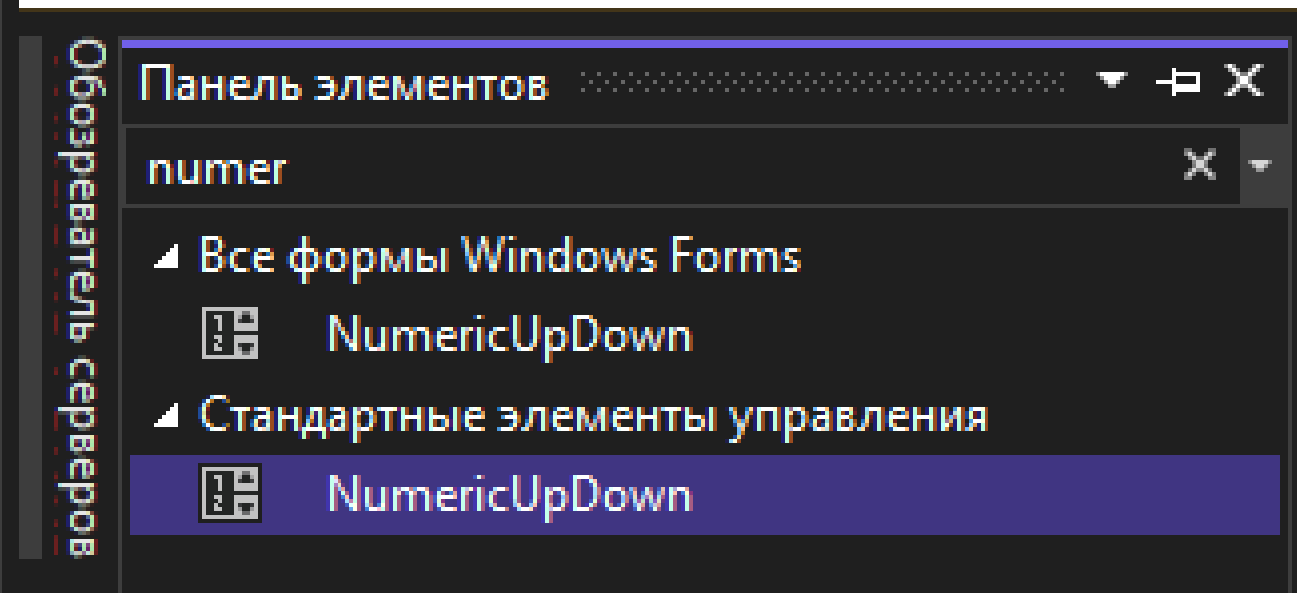
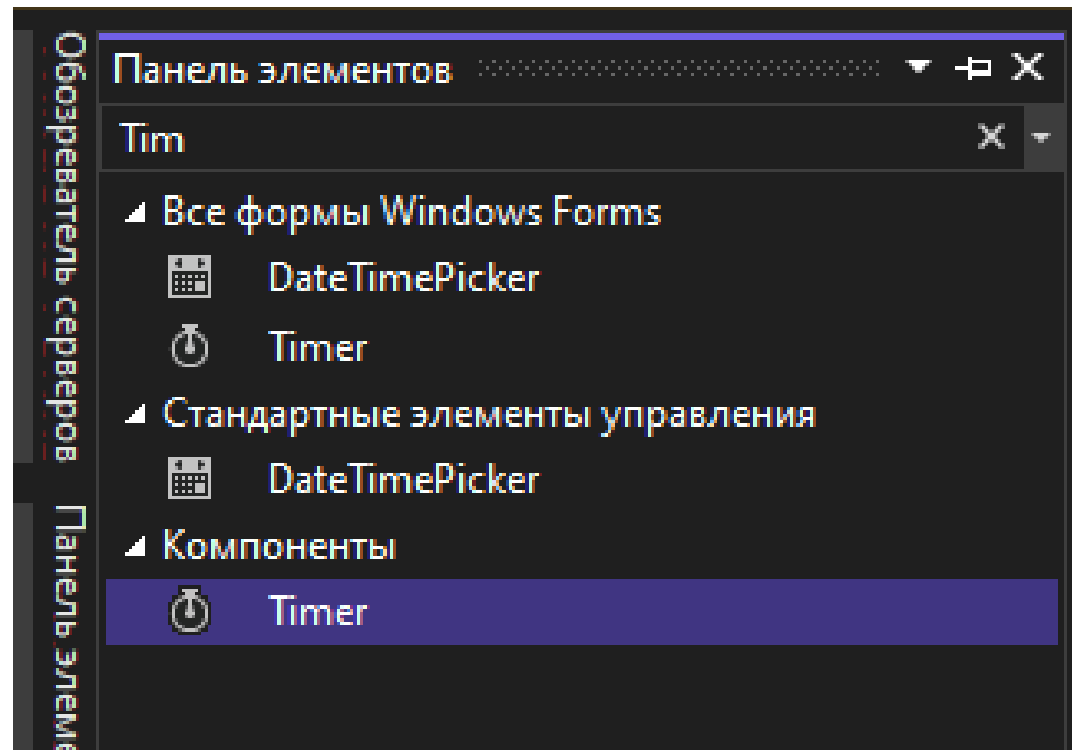
Name: button_start
Text: Start



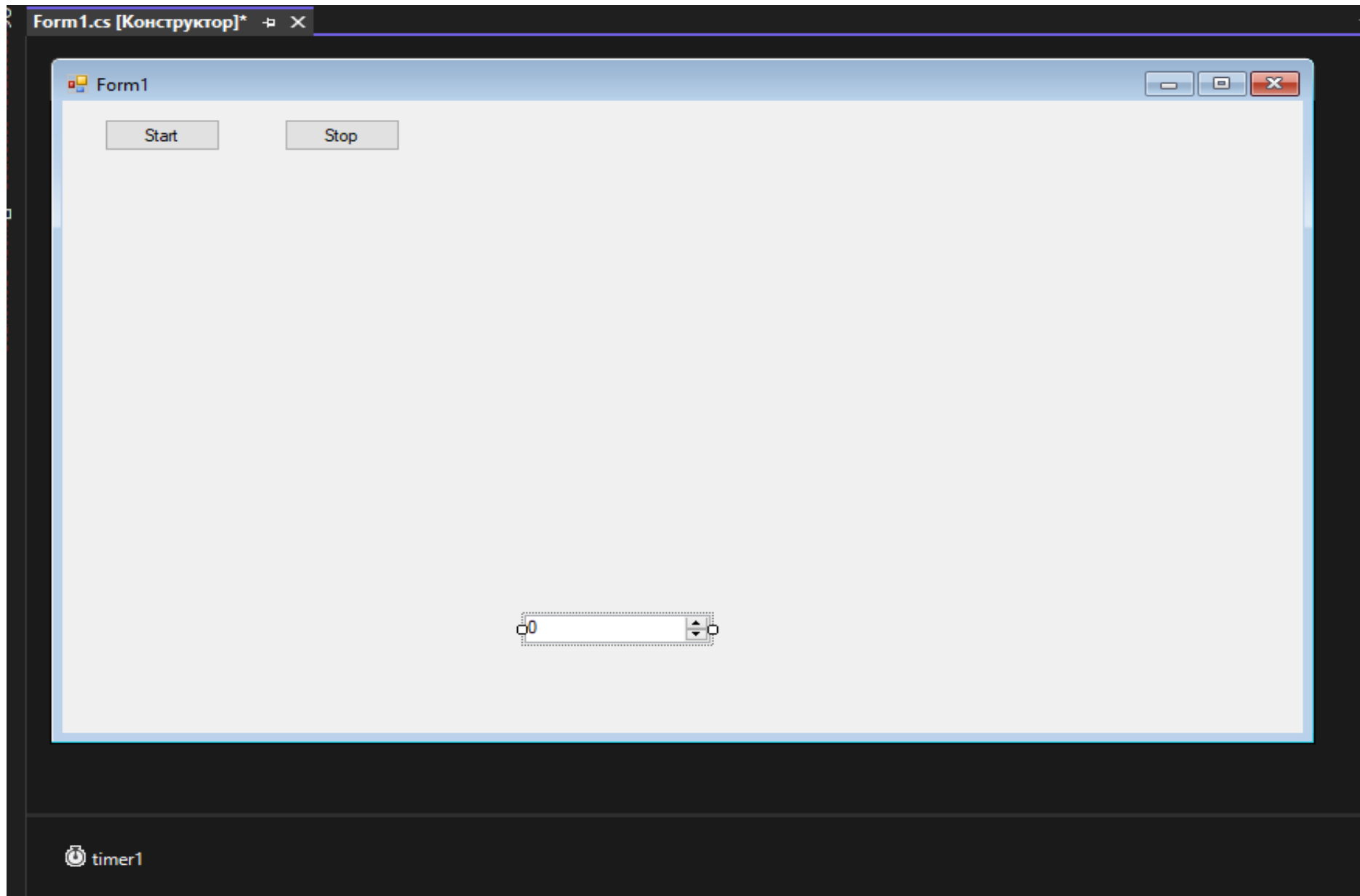
Именяем свойства кнопки
button1



Вставка «Timer» и «numericUpDown»



Здесь приведена форма с элементами управления



Увеличим шрифт на элементах управления, используя свойство «Font»

The image shows a Windows Forms application in Visual Studio. The main window, titled 'Form1', contains two buttons labeled 'Start' and 'Stop'. Below them is a numeric up-down control displaying the number '0'. A 'Шрифт' (Font) dialog box is open, showing the font selection process. The font is set to 'Microsoft Sans Serif', style to 'обычный' (normal), and size to '20'. The 'Набор символов' (Character set) is set to 'Кириллица' (Cyrillic). The 'Образец' (Preview) shows the text 'АаВвБбФф'. To the right, the 'Свойства' (Properties) window is open for the 'numericUpDown1' control. The 'Font' property is highlighted, showing 'Microsoft Sans Serif; 20,25p'. Below the properties window, a description of the 'Font' property is provided.

Шрифт

Шрифт: Microsoft Sans Serif
Начертание: обычный
Размер: 20

Видоизменение
☐ Зачеркнутый
☐ Подчеркнутый

Образец
АаВвБбФф

Набор символов: Кириллица

Свойства

numericUpDown1 System.Windows.Forms.NumericUpDown

Anchor	Top, Left
AutoSize	False
BackColor	Window
BorderStyle	Fixed3D
CausesValidation	True
ContextMenuStrip	(нет)
Cursor	Default
DecimalPlaces	0
Dock	None
Enabled	True
Font	Microsoft Sans Serif; 20,25p
ForeColor	WindowText

Font
Шрифт, используемый для отображения текста на элементе управления.

Объявим переменные, областью видимости которых является форма

```
public partial class Form1 : Form  
{
```



Это есть в шаблоне

```
Graphics g; // Объявляем графическую поверхность  
Rectangle rct; // Прямоугольник с размерами рисунка  
private Image Ball; // Рисунок  
int x = 400, y = 100, dx = 2, dy = 2;
```



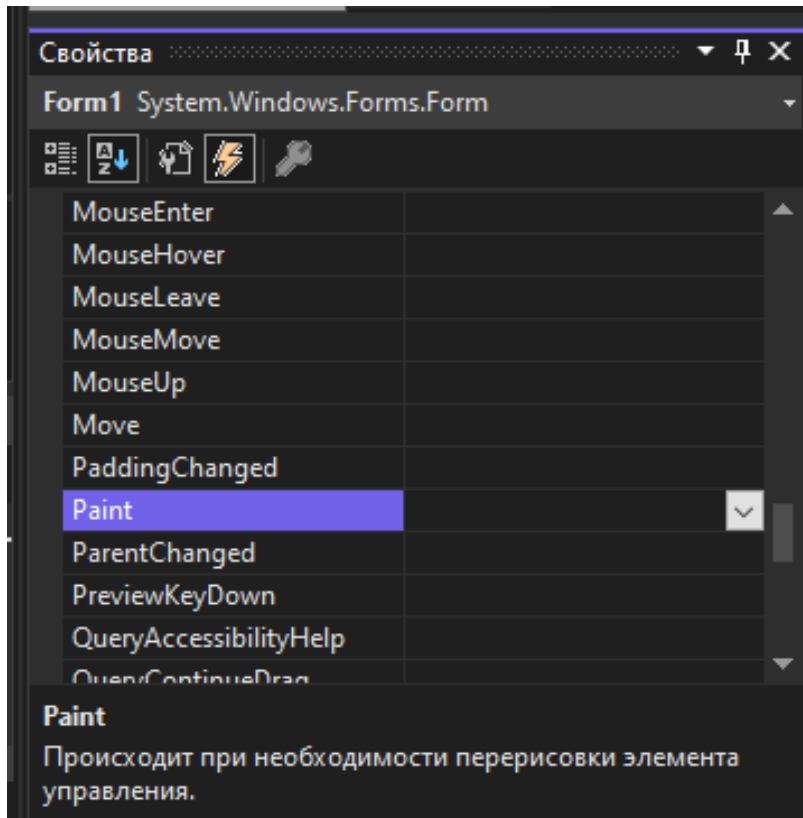
Это мы
вставляем

Получается

```
namespace WindowsFormsAppBall
{
    public partial class Form1 : Form
    {
        Graphics g; // Объявляем графическую поверхность
        Rectangle rct; // Прямоугольник с размерами рисунка
        private Image Ball; // Рисунок
        int x = 400, y = 100, dx = 2, dy = 2;

        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

Для формы создаем обработчик события Paint, чтобы нарисовать мяч



```
private void Form1_Paint(object sender,
PaintEventArgs e)
{
    g.Clear(Color.White); //Окрашиваем форму
    g.DrawImage(Ball, x, y,
Ball.Width, Ball.Height); //Рисуем мяч в
начальном положении
}
```

При инициализации формы вставляем рисунок, переменной Ball присваиваем значение

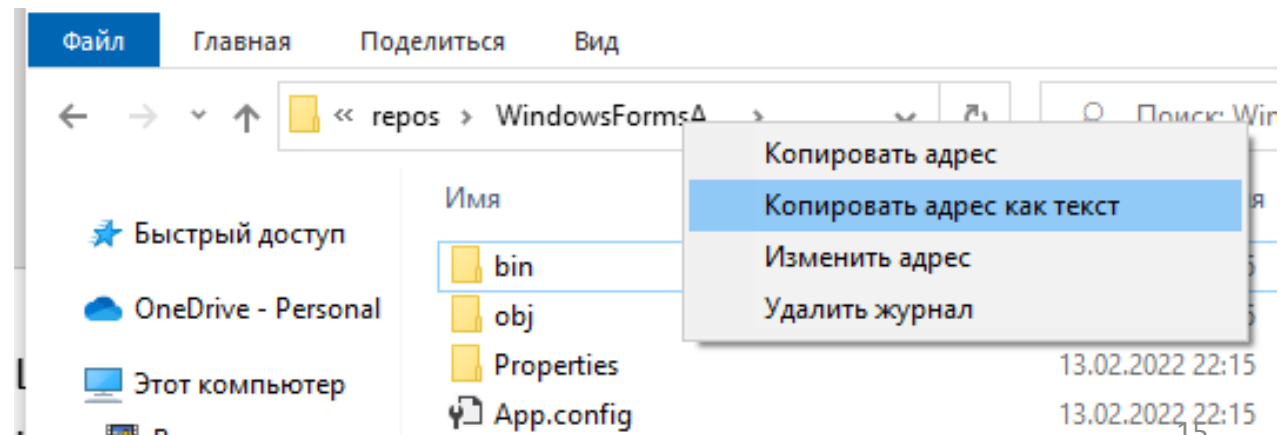
```
public Form1()  
{  
    InitializeComponent();
```

```
    Ball = new Bitmap("C:  
\\Users\\Administrator\\source\\  
\\repos\\WindowsFormsAMyach\\bal  
l.png"); //Создаем рисунок мяча  
}
```

← Это есть в шаблоне

← Это мы вставляем

Для определения пути к файлу рисунка используем «Копировать адрес как текст»

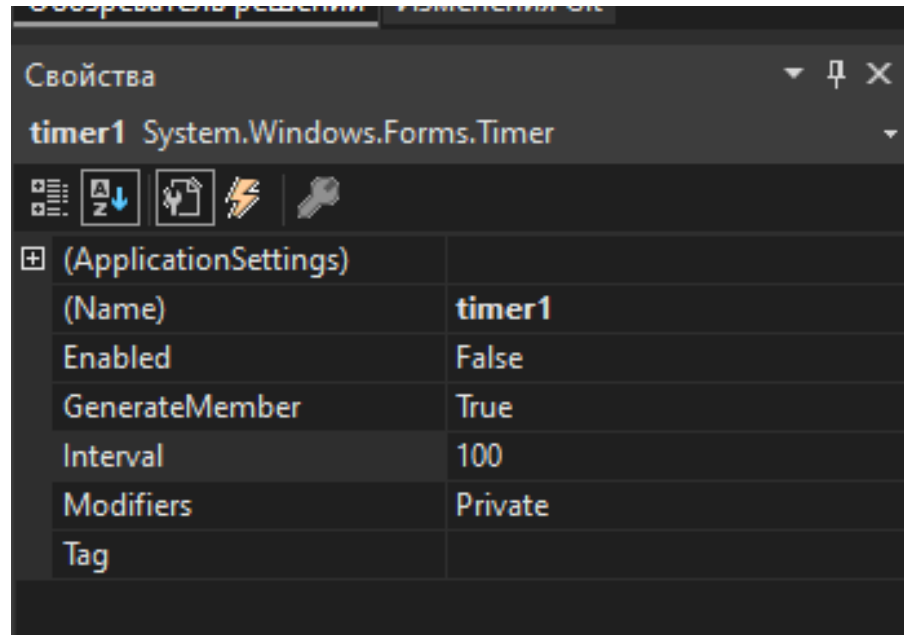


Получилось

```
public Form1()
{
    InitializeComponent();
    g = this.CreateGraphics();//Создаем поверхность
    Ball = new Bitmap("C:
\\Users\\Administrator\\source\\repos\\WindowsFormsAMyach\\ball.png");//Созда
ем рисунок мяча
}
```

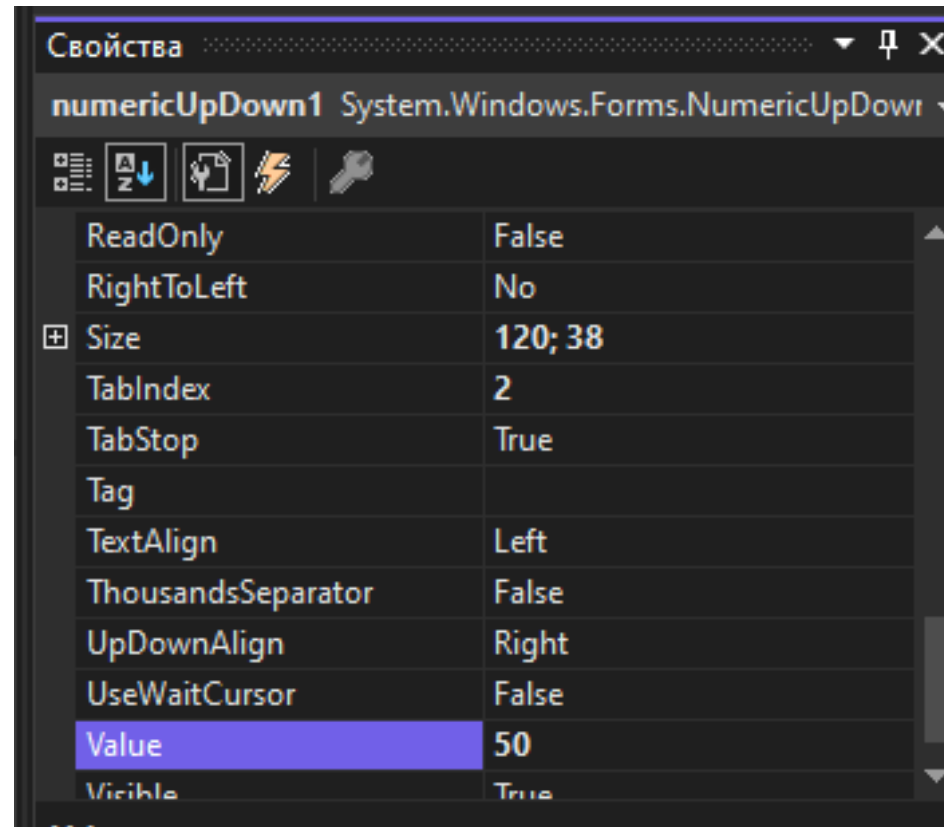
// К каждому обратному слэшу добавляем еще один обоатный слэш

Посмотрим свойства элемента Timer



Пока мы не редактировали **numericUpDown**, свойства Timer можно изменять, в частности Interval

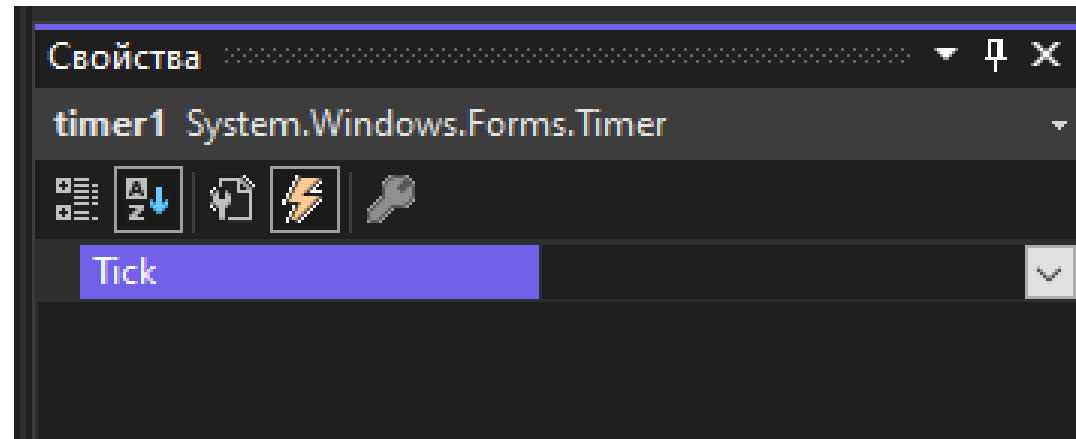
Теперь, поскольку мы хотим управлять Timer через **numericUpDown**, **Interval** его изменения будет указываться в его свойстве **Value**. Этой величине надо присвоить ненулевое значение, желательно больше 1 (иначе вариантов изменения не будет). После этого свойства **Timer** будут недоступны.



Создаем обработчик события Click на кнопке button_start

```
• private void button_start_Click(object sender, EventArgs e)
•     {
•         timer1.Interval = (int)numericUpDown1.Value; //Интервал таймера
•         timer1.Enabled = true; //Пуск таймера
•     }
```

Устанавливаем обработчик события Tick для Timer –это смещение мячика на один шаг по x и по y за интервал изменения таймера



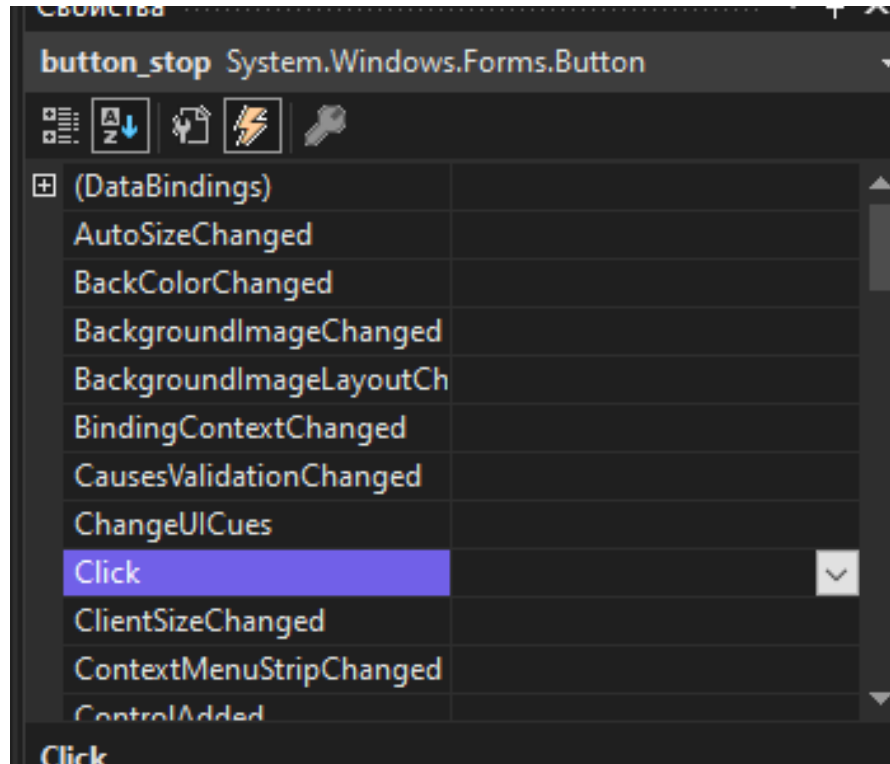
Вставляем текст обработчика Tick

```
private void timer1_Tick(object sender, EventArgs e)
{
    rct.X = x; //Создаем прямоугольник
    rct.Y = y;
    rct.Width = Ball.Width;
    rct.Height = Ball.Height;
    Invalidate(rct); //Перерисовываем прямоугольник. Стираем мяч
                        //Invalidate очищает прямоугольник. Если скобки пустые – всю
форму.
    x += dx; //Меняем координаты мяча
    y += dy;
    g.DrawImage(Ball, x, y, Ball.Width, Ball.Height); //рисует мяч в новом
положении
                                                //Отражение от стенок
    if ((x < 0) || (x > (ClientRectangle.Width - Ball.Width))) dx = -dx;
    if ((y < 0) || (y > (ClientRectangle.Height - Ball.Height))) dy = -dy;
}
```

Запускаем приложение и видим... Мяч можно, но скорость его движения постоянна. Он пока не останавливается



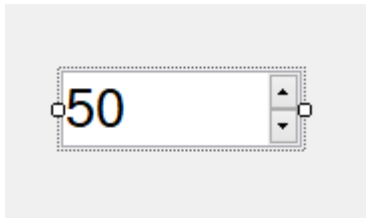
Вставим обработчик события Click для кнопки Stop. Теперь мяч
МОЖНО ОСТАНОВИТЬ



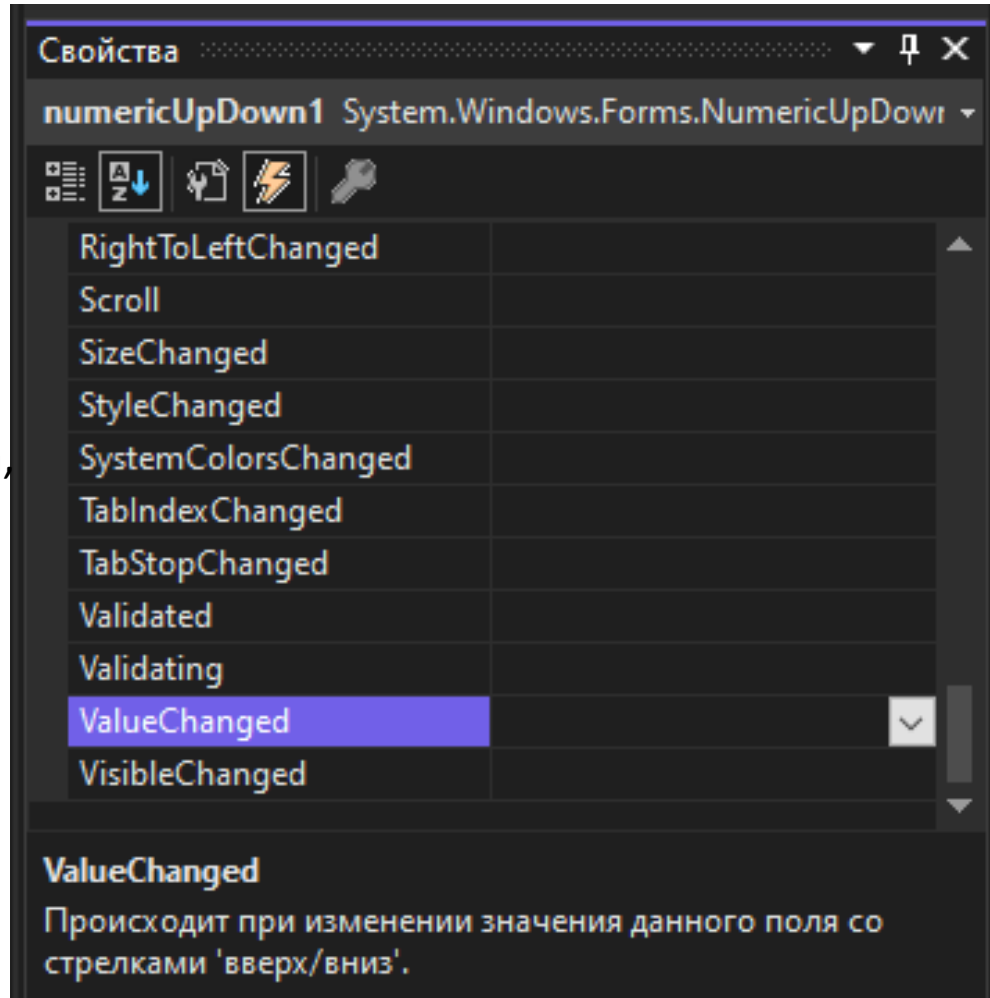
```
private void  
button_stop_Click(object sender,  
EventArgs e)  
{  
    timer1.Enabled =  
false; //Остановка таймера  
}
```

Теперь мяч можно остановить

А теперь создадим обработчик события «Изменение значения» (**ValueChanged**) для **numericUpDown1** – изменение интервала таймера



Чем больше интервал таймера, тем медленнее движется мяч



```
private void
numericUpDown1_ValueChanged(object
sender, EventArgs e)
{
    if
(((int)numericUpDown1.Value > 1) &&
((int)numericUpDown1.Value < 100))
    {
        timer1.Interval =
(int)numericUpDown1.Value; //Интервал
таймера
    }
}
```


Теперь можно

- Запустить мяч
- Остановить мяч
- Изменить его скорость